

TP2 — Pipeline data temps réel & plateforme data

Prolongement du TP1 — Immobilier × Énergie, Loire-Atlantique (44)

Louis Valette

Alexis Fouquet

Contents

1. Contexte et architecture	1
2. Sources	2
3. Logique d'agrégation	3
4. Data lake	3
5. Nettoyage PySpark	3
6. Observabilité (Prometheus + Grafana)	4
7. Data visualization	4
8. Résultats de la vérification	4

1. Contexte et architecture

Le TP1 a construit une base PostgreSQL statique (import unique, département 44) croisant transactions immobilières (DVF) et diagnostics de performance énergétique (DPE ADEME). Le TP2 prolonge ce travail avec un pipeline **temps réel** : ingestion via Kafka, data lake brut, nettoyage PySpark, rechargement continu dans la même base PostgreSQL, supervision technique et dashboard métier.

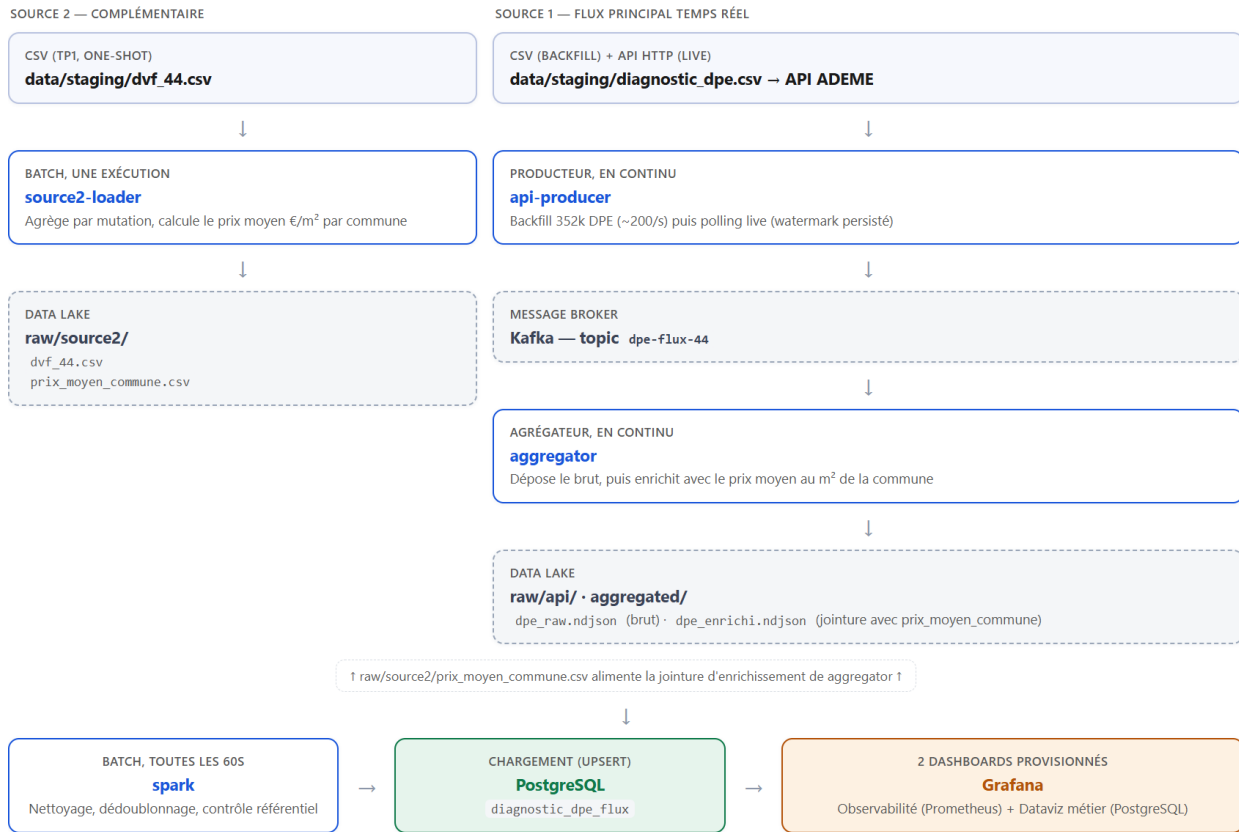


Figure 1: Architecture du pipeline temps réel TP2

2. Sources

#	Source	Nature	Rôle
1	API ADEME dpe03existant (diagnostics de performance énergétique)	API HTTP, alimentée sur Kafka par api-producer	Flux principal temps réel
2	DVF (transactions immobilières, département 44 — déjà utilisée en TP1)	CSV, chargée en un lot par source2-loader	Source complémentaire, sert à enrichir le flux 1

Pourquoi ce choix : les deux sources et le lien entre elles reprennent exactement la problématique du TP1 (marché immobilier × performance énergétique), pour rester cohérent avec la modélisation déjà posée. Réutiliser DVF évite une nouvelle source à qualifier de zéro et permet de vérifier la cohérence des deux TP par recoupement des mêmes indicateurs (section 8).

Pourquoi une phase de « backfill » : l'API ADEME ne produit que quelques nouveaux DPE par semaine pour le seul département 44 — insuffisant pour observer un flux en conditions de démo courte. api-producer rejoue donc d'abord les 352k DPE déjà collectés en TP1 à débit maîtrisé (200 événements/s, réglable via BACKFILL_RATE_PER_SEC), marqués source_flux=backfill, puis bascule en interrogation continue de l'API réelle (source_flux=live, toutes les POLL_INTERVAL_SECONDS) à partir d'un filigrane (watermark) persisté sur disque. Le backfill ne s'exécute qu'une fois : un marqueur (/state/backfill_done) évite de le rejouer à chaque redémarrage du conteneur.

3. Logique d'agrégation

aggregation/aggregator.py consomme le topic Kafka dpe-flux-44, dépose chaque événement tel quel dans raw/api/dpe_raw.ndjson (couche brute, non filtrée), puis l'enrichit avec le prix moyen au m² de la commune du diagnostic (calculé par source2-loader à partir de DVF, avec la même correction que le TP1 : agrégation par mutation avant calcul du prix/m², pour ne pas compter plusieurs fois la valeur d'une mutation à plusieurs lots) et dépose le résultat dans aggregated/dpe_enrichi.ndjson.

Justification métier : un DPE seul indique une étiquette énergétique ; associé au prix moyen au m² de sa commune, il permet de resituer un logement par rapport à son marché local — c'est la même question que celle du TP1 (est-ce que les logements F-G se vendent moins cher ?), mais évaluée en continu plutôt que sur un instantané.

4. Data lake

Le sujet propose une organisation MinIO (S3). **MinIO a fermé ses images Docker officielles derrière une authentification** (Docker Hub et quay.io renvoient tous deux 401 au moment du TP), ce qui rend un docker compose up reproductible impossible sans compte préalable. Le data lake est donc un volume Docker nommé (datalake_data), organisé exactement selon l'arborescence donnée par l'énoncé :

raw/api/	dpe_raw.ndjson	– flux Kafka brut, non filtré
raw/source2/	dvf_44.csv, prix_moyen_commune.csv	
aggregated/	dpe_enrichi.ndjson	– flux enrichi, avant nettoyage

5. Nettoyage PySpark

spark/jobs/clean_and_load.py tourne en boucle (local[*], pas de cluster nécessaire), relit aggregated/dpe_enrichi.ndjson toutes les 60 secondes et applique :

- contrôle de type (surfaces, prix, dates) ;
- rejet des étiquettes DPE/GES hors A-G ;
- rejet des lignes sans identifiant ou commune ;
- déduplication sur numero_dpe ;
- rejet des communes hors référentiel (jointure avec la table commune du TP1, réutilisée telle quelle).

Le résultat est chargé dans PostgreSQL par upsert (INSERT ... ON CONFLICT (numero_dpe) DO NOTHING, via psycopg2) dans la table diagnostic_dpe_flux (sql/06_create_flux_table.sql), dont la clé étrangère code_insee → commune garantit la cohérence avec le modèle du TP1.

Choix technique : chargement via psycopg2 plutôt que le writer JDBC de Spark, pour éviter de gérer un driver JDBC PostgreSQL supplémentaire dans l'image — inutile ici vu le volume (dizaines de milliers de lignes par cycle, pas un usage big data).

Robustesse : PostgreSQL n'exécute les scripts d'initialisation qu'à la toute première création d'un volume vide. Pour garantir diagnostic_dpe_flux quel que soit l'état du volume (par exemple un volume déjà initialisé par le TP1, avant l'existence de cette table), un service dédié (flux-table-init, CREATE TABLE IF NOT EXISTS, avec retry) s'exécute avant Spark à chaque démarrage — testé en reproduisant le cas exact (table supprimée manuellement sur un volume TP1 existant, puis docker compose up -d sans réinitialisation : la table est recrée automatiquement).

6. Observabilité (Prometheus + Grafana)

Cible	Exportateur	Ce qu'il expose
Conteneurs (CPU, mémoire, réseau)	cAdvisor	Infra — remplace node-exporter, écarté du périmètre : le TP tourne sur une seule machine, pas un cluster à superviser au niveau host
PostgreSQL	postgres-exporter	Connexions actives, disponibilité, taille de la base
api-producer, aggregator, spark	prometheus_client (Python), un exporteur HTTP par service	Métriques métier du pipeline

Indicateur Raw vs Clean (obligatoire) : `raw_records_total` (exposé par aggregator, volume déposé dans `raw/api/`) comparé à `clean_records_total` (exposé par spark, `COUNT(*)` réel sur `diagnostic_dpe_flux`). Les deux valeurs partent du volume réellement persisté (fichier / base), pas d'un compteur en mémoire remis à zéro au redémarrage, pour rester comparables dans la durée. `raw ≥ clean` en permanence : l'écart mesure le nettoyage (données rejetées) et le retard du cycle Spark (60s) sur le flux Kafka.

Dashboard Grafana provisionné automatiquement (aucune étape manuelle) : [monitoring/grafana/provisioning/dashboards/tp2_overview.json](#), 8 panels (Raw vs Clean, taux de nettoyage, débit du producteur par source, connexions/disponibilité/taille PostgreSQL, CPU/mémoire par conteneur).

7. Data visualization

Le sujet demande un dashboard de data visualization sans imposer d'outil. Plutôt qu'un second outil dédié (ex. Metabase), la dataviz métier est un second dashboard Grafana ([monitoring/grafana/provisioning/dashboards/tp2_metier.json](#)), branché sur un second datasource PostgreSQL (en plus du datasource Prometheus utilisé pour l'observabilité) : répartition des étiquettes DPE reçues, origine backfill/live, volume traité dans le temps, communes les plus chères, part de logements F-G par tranche de prix, derniers DPE reçus. Provisionné au même titre que le dashboard technique — pas d'étape manuelle, alors qu'un outil séparé (Metabase notamment) demande de construire son dashboard à la main dans son UI au premier lancement.

8. Résultats de la vérification

Stack testée de bout en bout à froid (`docker compose down -v && docker compose up -d`), sur trois machines et réseaux différents.

Étape	Résultat observé
Source 2 (DVF)	Chargée en un lot, prix moyen calculé pour 207 communes, 0 erreur
Table <code>diagnostic_dpe_flux</code>	Créée automatiquement, y compris sur un volume déjà initialisé par le TP1
Kafka (<code>dpe-flux-44</code>)	Backfill complet : 351 959 événements publiés
Data lake	<code>raw/api/</code> et <code>aggregated/</code> : mêmes volumes en parallèle (cohérence vérifiée)
PostgreSQL (<code>diagnostic_dpe_flux</code>)	351 959 lignes chargées après backfill complet
Cohérence Raw vs Clean	Vérifiée à la source (fichiers + base, pas seulement les métriques) : <code>raw ≥ clean</code> à tout instant

Étape	Résultat observé
Prometheus	5/5 cibles UP (cadvisor, postgres, api-producer, aggregator, spark)
Grafana	2 dashboards et 2 datasources (Prometheus, PostgreSQL) provisionnés automatiquement ; requêtes testées via l'API de Grafana avec des données réelles

Le classement des communes les plus chères, recalculé en continu par le flux temps réel, retombe sur les mêmes valeurs que le TP1 (La Baule-Escoublac 6 536 €/m², Pornichet 5 636 €/m², Nantes 3 777 €/m²) — confirmation croisée entre les deux TP.

Comment reproduire

```
docker compose up -d
```

Démarré l'ensemble (TP1 + TP2) : base PostgreSQL et son import TP1, data lake, Kafka, le pipeline complet (producteur, agrégateur, Spark), Prometheus, Grafana et cAdvisor — sans étape manuelle ni accès réseau obligatoire (le backfill utilise les CSV déjà versionnés ; seule la phase « live » de api-producer appelle l'API ADEME, avec repli silencieux si elle est inaccessible).

Comment vérifier

Composant	Comment vérifier
Kafka	<code>docker exec tp2_kafka /opt/kafka/bin/kafka-get-offsets.sh --bootstrap-server localhost:19092 --topic dpe-flux-44</code> → nombre de messages produits
Data lake	<code>docker exec tp2_aggregator sh -c "wc -l /datalake/raw/api/dpe_raw.ndjson /datalake/aggregated/dpe_enrichi.ndjson"</code>
PostgreSQL (flux)	<code>docker exec tp_audit_44_pg psql -U postgres -d audit_immo_energie_44 -c "SELECT COUNT(*) FROM diagnostic_dpe_flux;"</code>
Prometheus	<code>http://localhost:9090/targets</code> — les 5 cibles doivent être UP
Grafana	<code>http://localhost:3000</code> (admin/admin, ou accès anonyme activé) — dashboards <i>TP2 — Pipeline temps réel</i> et <i>TP2 — Dataviz métier</i>
Métriques brutes	<code>http://localhost:8001/metrics</code> (producteur), <code>http://localhost:8002/metrics</code> (agrégateur), <code>http://localhost:8003/metrics</code> (Spark)

Ports exposés

Port	Service
5432	PostgreSQL
9092	Kafka (accès externe)
8001 / 8002 / 8003	Métriques Prometheus : producteur / agrégateur / Spark
8080	cAdvisor
9187	postgres-exporter

Port	Service
9090	Prometheus
3000	Grafana